



Mapa Turístico

Matemáticas discretas
501252-1

Integrantes: Rodrigo Domínguez Larenas

Ariel Fernández Fuentealba

Felipe Grandón Contreras

Profesora: Lilian Angélica Salinas Ayala



Índice

Introducción	3
Objetivos	4
Objetivo general	4
Objetivos específicos	4
Modelo	5
1.1 Modelo	5
1.2. Lectura del mapa desde coordenadas	5
1.3. De línea geométrica a vértices del grafo	5
1.4. Intersecciones como vértices	6
1.5. Construcción de aristas y pesos	6
1.6. Búsqueda de rutas con Dijkstra	7
1.7. Síntesis del modelo	7
1.8. Exploración adicional, optimización de caminos y versión gráfica	8
Implementación	9
1. Estructuras de Datos y Tipos Definidos	9
2. Motor de Procesamiento Geométrico	9
3. Construcción del Grafo y Gestión de Memoria	9
4. Algoritmo de Búsqueda de Rutas	10
5. Lógica de Salida y Agrupación de Instrucciones	10
Conclusión	11



Introducción

Este informe presenta el desarrollo de un programa en C para modelar el mapa turístico de una ciudad, apoyándose en conceptos de teoría de grafos. La idea central es representar calles y puntos de interés con estructuras de datos adecuadas, para luego calcular rutas que permitan recorrer los lugares turísticos definidos.

Los mapas permiten orientarse, ubicar calles y planificar recorridos entre distintos lugares. Sin embargo, para que un computador pueda trabajar con un mapa, esta información debe representarse mediante datos y estructuras que permitan identificar conexiones, cruces y posibles caminos. En este contexto, la teoría de grafos entrega una herramienta adecuada para modelar relaciones entre puntos y analizar rutas dentro de un espacio determinado.

El programa parte de un archivo de entrada con información sobre las calles, sus coordenadas y la ubicación de los puntos turísticos. Con esos datos se construye el mapa como un grafo, donde las intersecciones actúan como nodos y las conexiones entre calles son las aristas. Esa estructura es la que hace posible encontrar caminos entre los distintos puntos turísticos y armar una ruta válida para visitarlos.

Durante el desarrollo también se trabajó en el manejo de errores y la validación de entradas, lo que le da mayor estabilidad al programa y evita que falle en medio de la ejecución. En general, el proyecto fue una buena oportunidad para aplicar y afianzar contenidos como estructuras de datos, modelamiento de problemas y algoritmos de búsqueda, además de entender mejor cómo estas ideas pueden trasladarse a situaciones parecidas a problemas del mundo real.



Objetivos

Objetivo general

- Diseñar e implementar un programa en lenguaje C que modele un mapa turístico mediante un grafo ponderado, a partir de calles definidas por coordenadas y puntos de interés, para calcular rutas factibles entre los lugares turísticos registrados en el archivo de entrada.

Objetivos específicos

- Leer desde un archivo la información de calles, coordenadas, eje de numeración y puntos turísticos.
- Validar que los datos ingresados sean coherentes, evitando calles inválidas, coordenadas fuera de rango o puntos ubicados fuera de su calle correspondiente.
- Transformar las calles del mapa en segmentos dentro de un plano cartesiano, identificando extremos, intersecciones y puntos turísticos como vértices del grafo.
- Construir un grafo no dirigido y ponderado, conectando los vértices consecutivos de cada calle mediante aristas cuyo peso corresponde a la distancia del tramo.
- Aplicar el algoritmo de Dijkstra para calcular caminos de menor distancia entre puntos turísticos.
- Mostrar por consola un recorrido turístico comprensible, indicando los tramos recorridos, las calles utilizadas y la distancia total.



Modelo

1.1 Modelo

El programa modela un mapa turístico a partir de un archivo de entrada que contiene calles y puntos turísticos. Cada calle se entrega mediante coordenadas iniciales y finales, por lo que el programa interpreta cada una como un segmento dentro de un plano cartesiano. A partir de esos segmentos se construye un grafo, donde los vértices representan lugares relevantes del mapa y las aristas representan tramos transitables entre ellos.

1.2. Lectura del mapa desde coordenadas

El archivo de entrada comienza indicando la cantidad de calles. Luego, cada calle se describe con seis datos: nombre, coordenada inicial, coordenada final y eje de numeración. El formato usado por el programa es el siguiente:

Nombre_calle	x1	y1	x2	y2	Eje
Los_carrera	0	50	1200	50	X
Freire	0	200	1200	200	X
Janequeo	650	0	650	800	Y
Tucapel	900	0	900	800	Y

Por ejemplo, la línea “Los_Carrera 0 50 1200 50 X” significa que la calle Los_Carrera comienza en la coordenada (0,50) y termina en la coordenada (1200,50). Como su eje de numeración es X, un punto turístico ubicado en la posición 300 sobre esa calle se interpreta como la coordenada (300,50).

Dato leído	Función dentro del modelo	Ejemplo
x1, y1	Coordenada inicial de la calle.	(0,50)
x2, y2	Coordenada final de la calle.	(1200,50)
eje X o Y	Indica cómo ubicar puntos turísticos dentro de la calle.	X: se usa la posición como coordenada x
nombre de calle	Permite asociar puntos turísticos a una calle existente.	Museo Los_Carrera 300

1.3. De línea geométrica a vértices del grafo

Una vez leídas las calles, el programa no crea inmediatamente una única figura cerrada ni un cuadrado. Lo que hace es considerar cada calle como una línea o segmento de recta. Sobre cada segmento identifica puntos importantes que luego se transformarán en vértices del grafo.

Los vértices principales que se generan son los siguientes:

- **Extremos de las calles:** corresponden al punto inicial y final de cada segmento.
- **Intersecciones:** si dos calles se cruzan, el punto de cruce se agrega como vértice.
- **Puntos turísticos:** cada lugar turístico se convierte en un vértice porque puede ser origen, destino o punto visitado durante el recorrido.

En el código, esta lógica se observa en la función `construir_grafo()`, donde para cada calle se agregan los parámetros 0.0 y 1.0, que representan el inicio y el final del segmento. Luego se agregan los parámetros correspondientes a intersecciones y puntos turísticos ubicados sobre esa misma calle. Después se ordenan esos puntos y se conectan los consecutivos.

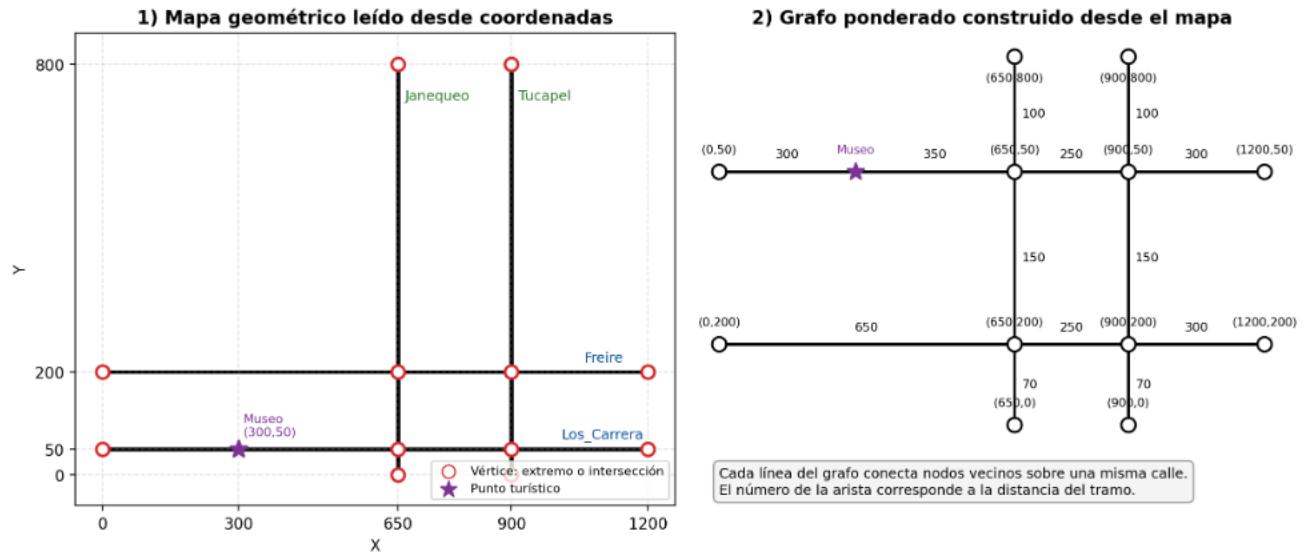


Figura 1. Transformación de calles por coordenadas en vértices y aristas de un grafo.

1.4. Intersecciones como vértices

Las intersecciones son fundamentales porque permiten que el recorrido cambie de una calle a otra. Si una calle horizontal y una calle vertical se cruzan, el punto de cruce se incorpora al grafo como un vértice compartido. Esto permite que el algoritmo entienda que desde ese punto se puede continuar por cualquiera de las calles conectadas.

(tomaremos de ejemplo nuestro `input3.txt`)

Calle 1	Calle 2	Vértice generado
Los_Carrera	Janequeo	(650,50)
Los_Carrera	Tucapel	(900,50)
Freire	Janequeo	(650,200)
Freire	Tucapel	(900,200)

El programa evita duplicar vértices mediante la función `agregar_nodo()`. Si un vértice con la misma coordenada ya existe, reutiliza ese índice en lugar de crear otro nodo. Gracias a esto, una intersección queda representada como un único punto del grafo, aunque pertenezca a dos calles distintas.

1.5. Construcción de aristas y pesos

Después de detectar los vértices de cada calle, el programa los ordena según su ubicación dentro del segmento. Luego conecta cada vértice con el siguiente vértice de la misma calle. Esa conexión se denomina arista.



Por ejemplo, si sobre una misma calle existen los puntos (0,50), (300,50), (650,50), (900,50) y (1200,50), el programa no une directamente el primero con el último. Primero divide la calle en tramos menores y conecta solo nodos vecinos:

(0,50) ↔ (300,50)

(300,50) ↔ (650,50)

(650,50) ↔ (900,50)

(900,50) ↔ (1200,50)

El peso de cada arista corresponde a la distancia geométrica entre los dos vértices conectados. Como los tramos se modelan en un plano cartesiano, la distancia se calcula con la fórmula euclidiana:

$$\text{Distancia} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Además, las aristas se agregan en ambos sentidos. Por eso el grafo resultante es no dirigido: si se puede avanzar desde un nodo A hacia un nodo B, también se puede volver desde B hacia A por el mismo tramo.

1.6. Búsqueda de rutas con Dijkstra

Cuando el grafo ya está construido, el programa utiliza el algoritmo de Dijkstra para calcular el camino más corto entre dos puntos turísticos. Dijkstra trabaja sobre grafos ponderados (con pesos) y busca la ruta cuya suma de pesos sea menor.

En este programa, cada peso representa distancia. Por lo tanto, el camino elegido será el que tenga menor distancia total entre el punto turístico de origen y el punto turístico de destino.

El recorrido completo se realiza de manera secuencial: se parte desde el primer punto turístico, se avanza hacia el siguiente punto no visitado y se repite el proceso hasta marcar todos los puntos turísticos. Si durante un tramo se pasa por otro punto turístico, también se considera visitado.

1.7. Síntesis del modelo

Paso	Nombre	Descripción
1	Lectura	El programa lee calles y puntos turísticos desde el archivo
2	Geometría	Cada calle se interpreta como un segmento definido por coordenadas.
3	Vértices	Se crean vértices en extremos, intersecciones y puntos turísticos.
4	Aristas	Se conectan vértices consecutivos sobre cada calle.
5	Pesos	Cada arista recibe como peso la distancia entre sus dos vértices.
6	Ruta	Dijkstra calcula el camino más corto entre puntos turísticos.



Por lo tanto, el modelo usado por el programa consiste en transformar un mapa de calles, descrito mediante coordenadas, en un grafo ponderado. Las líneas del archivo se convierten en segmentos dentro de un plano cartesiano. Desde esos segmentos se generan vértices en los extremos, en las intersecciones y en los puntos turísticos. Luego, los vértices vecinos de una misma calle se conectan mediante aristas cuyo peso corresponde a la distancia del tramo.

Entonces el programa no construye el grafo simplemente “haciendo un cuadrado” como al principio suponíamos, sino que lo genera a partir de la geometría real entregada por el archivo. Que lee cada calle como una línea definida por coordenadas. Los extremos de esas líneas son vértices, y cuando dos calles se cruzan también se crea un vértice en la intersección. Luego conecta los vértices vecinos de cada calle, asigna como peso la distancia del tramo y con eso forma un grafo ponderado para calcular rutas

Esta estructura permite aplicar Dijkstra para buscar rutas cortas, ya que el mapa queda convertido en nodos, conexiones y distancias.

De esta manera, con este modelo el problema geométrico de recorrer calles se transforma en un problema clásico de grafos con Dijkstra y pasamos a la implementación.

1.8. Exploración adicional, optimización de caminos y versión gráfica

Además de la versión oficial por consola, se desarrollaron pruebas complementarias con una versión gráfica y con una estrategia alternativa para optimizar el orden de visita de los puntos turísticos. Esta versión no se considera la entrega principal, ya que incorpora elementos fuera del alcance de la pauta y, en algunos casos, funciones dependientes del entorno de ejecución.

La versión oficial mantiene el criterio solicitado: parte desde el primer punto turístico y avanza secuencialmente hacia el siguiente punto no visitado, usando Dijkstra para calcular el camino más corto entre cada par de puntos. En cambio, la versión exploratoria prueba distintos órdenes de visita mediante permutaciones, buscando reducir la distancia total del recorrido.

Este enfoque puede entregar recorridos más cortos, pero tiene una complejidad factorial $O(n!)$, por lo que su costo computacional aumenta rápidamente al crecer la cantidad de puntos turísticos. Por esa razón, se presenta como una mejora experimental y no como el modelo principal del programa.

En una prueba realizada con el ejemplo de la pauta, la versión secuencial obtuvo una distancia aproximada de 1146.42 unidades, mientras que la versión exploratoria obtuvo 876.42 unidades. Esto representa una disminución cercana al 23.55% en la distancia recorrida. Este resultado muestra que el orden de visita influye en la distancia total, aunque también evidencia que la optimización global requiere mayor costo computacional.



Implementación

La implementación del sistema se realizó en lenguaje C (estándar C11), priorizando una arquitectura modular y el uso eficiente de la memoria dinámica. El programa se descompone en cuatro pilares fundamentales: gestión de datos, motor geométrico, construcción de la topología y algoritmo de búsqueda.

1. Estructuras de Datos y Tipos Definidos

Se definieron estructuras personalizadas para representar fielmente el modelo matemático en el entorno computacional:

- **struct Calle:** Almacena los metadatos de los segmentos (nombre, coordenadas y eje de numeración). Es la base para la interpretación del archivo de entrada.
- **struct Nodo:** Representa un punto discreto en el plano (x, y). Esta abstracción permite que tanto las esquinas como los puntos turísticos sean tratados de forma genérica por el algoritmo de búsqueda.
- **struct AristaLista:** Es el componente básico de nuestra **Lista de Adyacencia**. Almacena el índice del nodo destino, el puntero al siguiente elemento y, fundamentalmente, una referencia al índice de la calle original para permitir la reconstrucción de instrucciones verbales.

2. Motor de Procesamiento Geométrico

El corazón de la implementación radica en cómo el programa traduce coordenadas en conexiones. Se implementaron funciones específicas para:

- **Cálculo de Intersecciones:** Utiliza un sistema de ecuaciones lineales para determinar si dos segmentos se cruzan. Se emplea una constante de tolerancia EPS (10^{-6}) para mitigar los errores de precisión de punto flotante, asegurando que los cruces se detecten correctamente incluso en coordenadas cercanas a los límites de los segmentos.
- **Parametrización de Nodos:** Cada nodo ubicado sobre una calle se representa mediante un parámetro t perteneciente a $[0, 1]$. Esto facilita el ordenamiento de los nodos mediante la función `qsort` de la librería estándar, garantizando que las aristas se creen solo entre nodos físicamente contiguos en el trazado de la calle.

3. Construcción del Grafo y Gestión de Memoria

El grafo se construye dinámicamente. Para optimizar el uso de recursos, se implementó una función `agregar_nodo()` que realiza una búsqueda previa para evitar la duplicidad de vértices en intersecciones compartidas.

La adyacencia se gestiona mediante memoria dinámica (`malloc`). Debido a la naturaleza del proyecto, donde se pueden cargar múltiples archivos en una sola sesión, la gestión del ciclo de vida de los datos es crítica.



Se diseñó un protocolo de liberación de memoria (`limpiar_adyacencia`) que recorre cada lista enlazada aplicando `free()`, previniendo fugas de memoria (*memory leaks*) que podrían comprometer la estabilidad del sistema en ejecuciones prolongadas.

4. Algoritmo de Búsqueda de Rutas

Se implementó el **Algoritmo de Dijkstra** para la obtención de caminos factibles. La lógica de búsqueda opera sobre tres arreglos principales:

- `dist[]`: Almacena la distancia mínima acumulada desde el origen.
- `visitado[]`: Un conjunto booleano que marca los nodos ya procesados.
- `padre[]`: Esencial para la reconstrucción del camino, guardando la trazabilidad de los nodos para generar la ruta final.

5. Lógica de Salida y Agrupación de Instrucciones

Para mejorar la usabilidad, el programa incluye una capa de procesamiento de salida que agrupa segmentos sucesivos pertenecientes a la misma calle. En lugar de emitir una instrucción por cada intersección, el sistema suma las distancias de tramos contiguos, generando una indicación única y clara (ej. "*Avanza 150 unidades por Calle X*"), lo que dota al programa de una interfaz de usuario por consola mucho más profesional y legible.



Conclusión

El desarrollo del programa permitió aplicar conceptos centrales de matemáticas discretas, especialmente teoría de grafos, a un problema representado inicialmente de forma geométrica. A partir de calles descritas por coordenadas, el programa transforma líneas del plano en vértices, aristas y pesos, logrando convertir un mapa turístico en una estructura adecuada para el cálculo de rutas.

El modelo implementado demuestra que los extremos de las calles, las intersecciones y los puntos turísticos son los elementos para construir el grafo. Los extremos permiten delimitar cada calle, las intersecciones permiten cambiar de una calle a otra y los puntos turísticos permiten definir los lugares que deben visitarse. Al conectar únicamente vértices consecutivos sobre cada calle, el programa conserva la forma real del mapa y evita crear conexiones que no existen en el trazado original.

El uso de Dijkstra resulta adecuado para este problema, ya que todas las aristas tienen pesos positivos asociados a la distancia entre nodos. De esta manera, el programa puede encontrar caminos de menor distancia entre puntos turísticos y entregar indicaciones comprensibles para el usuario.

Como limitación, la versión oficial no busca necesariamente el recorrido global más corto entre todos los puntos turísticos, ya que sigue el criterio secuencial definido por la pauta y el orden de entrada. Sin embargo, esta decisión cumple con el enfoque solicitado y mantiene una implementación clara, portable y consistente con los contenidos del curso. Como mejora futura, se podría incorporar una optimización del orden de visita o una visualización gráfica del mapa y del recorrido obtenido (como la entrega del programa exploratorio).

El proyecto permitió comprender un problema cotidiano, como recorrer puntos turísticos en una ciudad, puede ser este problema modelado mediante grafos ponderados y resuelto con algoritmos clásicos de búsqueda de caminos, Dijkstra en nuestro caso.